Lecture 16 : Designing Google Docs/Collaborative Editing

Google Docs :

- Q: What can user do?
  ↳ Single user doc editing
  Multi user same doc editing (Collaborative editing)
- Q: What is DAU?
  ↳ 5 million
- Q: How many concurrent user can edit doc at a time?
  ↳ 100 users

Functional Req :

- Single user editing : Single user can create/read/update /delete on a document.
- Multi user editing : Multiple user can edit same document at a time.

Non-functional Req :

1. Availability over Consistency.
2. Eventual consistency is accepted.
3. Read your own writes : user can see his/her edits immediately.

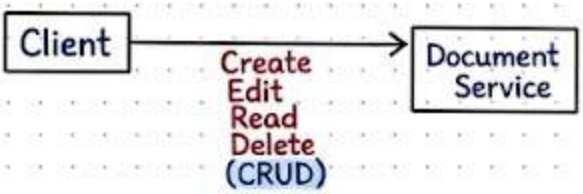
Non-focused area :

- Authentication & Authorization
- Access writes
- ....

↳ Back of the Envelope Calculations :

- QPS (Query per second)
- peak QPS = QPS * 2
- QPS = Read Query, Write Query (Read : write → 10 : 1)
- Storage unit : - Each doc roughly consume 100 KB.
  Estimate for 5 years.

↳ API Endpoints:



1. Create a new document

POST /add/document

Request Body:

```
{ title : "first document",
}
```

Response :

```
{ documentId : "123",
  title : " first document"
}
```

2. Read an existing doc.

GET /fetch/document/{documentId}

Response :

```
{ documentId : " ",
  title : " ",
  content : " ",
  created_at : " ",
  updated_at : " ",
}
```

3. Edit an existing document

PUT /edit/document/{documentId}

Request Body:

```
{ title : " ",
  content : " "
}
```

Response : { documentId : " ", title : " ", content : " " }

4. Delete an existing doc

Delete /remove/document/{documentId}

Response : 200 Ok

```
{ document Id : "" }
```

5. Editing within a document

POST /edit/document/{documentId}/operation

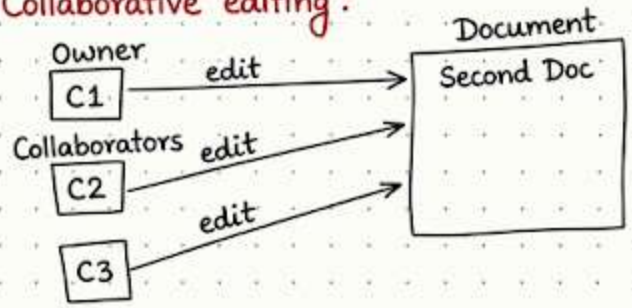
Request Body:

```
{ operation : [
        {operationId : "",
         type : "insert",
         position : "20"
         text : "Hello" },
        { }
    ]
}
```

Response : 200 Ok

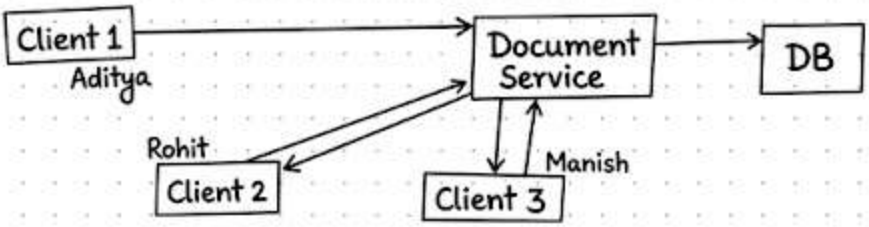
```
{ documentId : ""
  status : "success"
}
```

6. Collaborative editing :



Method 1. Using HTTP polling.

Request - Response Arch → polling



Not a good approach.

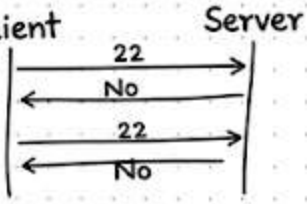
pros: Easy to implement

Cons:

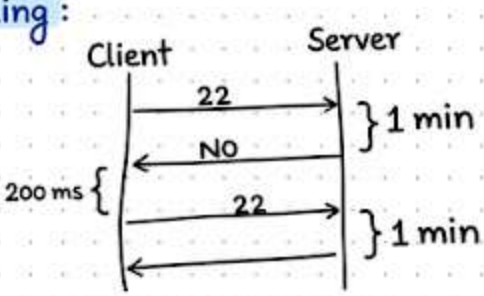
1. Server will be bombarded with too much requests.
2. Latency will increase (slow)
3. Bandwidth consume a lot of Bandwidth.

2. Long polling:

In short polling:



In long polling:



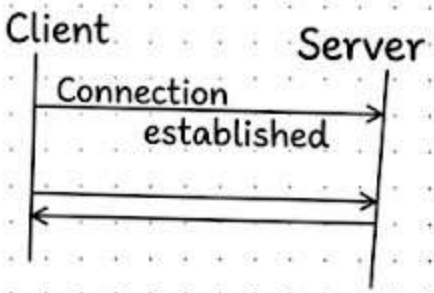
pros:

1. Server load is slightly reduced

Cons are the same

3. using Web Sockets

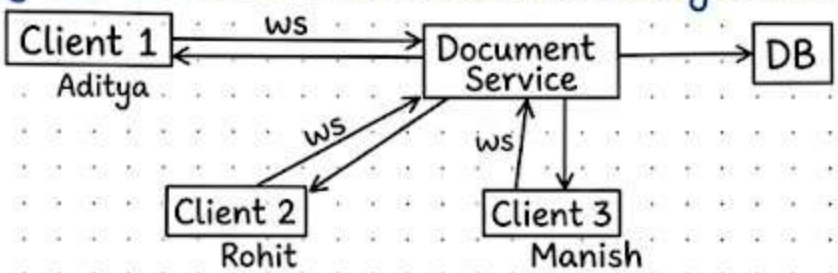




REST APIs vs WEB Sockets :

HTTP : POST, GET, PUT, DELETE etc

Why do we need Web sockets in Google docs?



So, 6. Collaborative editing.

WS /api/document/{documentId}/collaborate

Response : 200

```
{
  "message" : "Connection established"
}
```

WS /edit/document/{documentId}

Broadcast response to every connected user

Request Body :

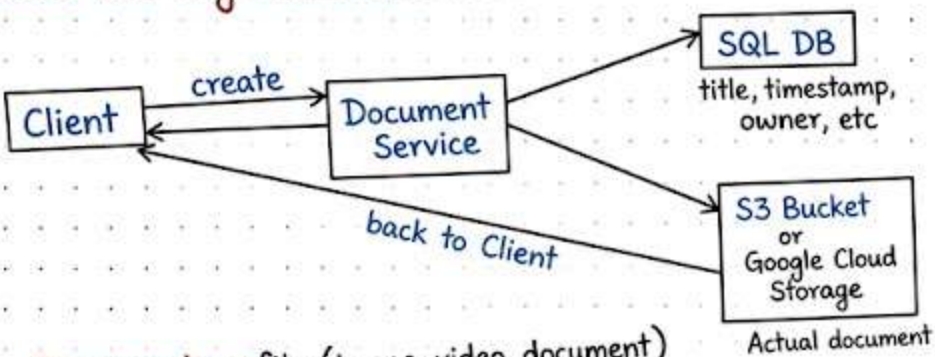
```
{ type : "operation",
  operation : {
        type : "Insert",
        position : 20,
        text : "New text"
  }
}
```

Response :

```
{ type : "operation",
  userId : "user-1",
  operation : {
      type : "Insert",
      position : 20,
      text : "Hello"
  }
}
```

pros:
↳ Latency will be low
↳ Server is not bombarded with request.

Draw the High level architecture:



Document → file (image, video, document)
Amazon S3 Buckets

4. Link sharing in Collaborative editing
https://www.amazons3.com/bucket/documentId/...

Hierarchical file system:
Windows, MacOS
Home → coding → HLD → HLD_Lec-1.pdf

↳ Hierarchical folder structure is not good approach for file sharing:
↳ Too long URL
↳ We don't want to expose entire folder hierarchy to all users

↳ Flatten up the URL
↳ generate a unique URL (unique ID) for each file shared

- Google Docs
- Notion

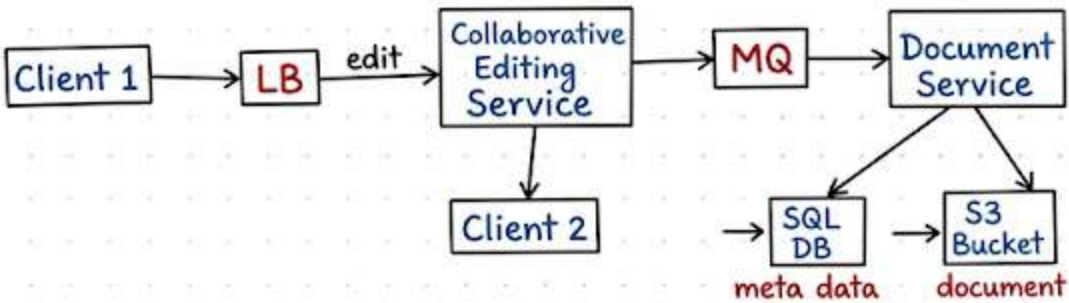
How to generate this unique Id ?

- Unique
- As short as possible

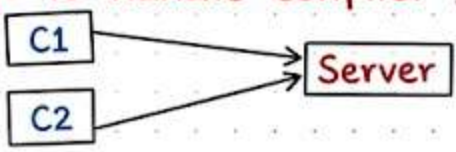
→ Google Docs → alphanumeric

→ Notion → MD5 algorithm (generate MD5 has of document name)

URL shortener → base X approach



↳ How to handle conflict ??



C1 : insert "hello" at pos 0.
C2 : insert "Bye" at pos 0.

↳ Basic approach :

1, Timestamp based approach

C1 → Amazing at pos 0
timestamp : 10:22:22

C2 → Brilliant at pos 0
timestamp : 10:22:23



2. Operational Transformation (OT)

- Use by google docs.